



# 《AI大模型Ragent项目》——文件上传分布式限流如何做？

来自： 拿个offer-开源&项目实战



马丁

2026年03月29日 12:40

前面两篇文章解决了文件上传的两个关键问题：通过 Spring Boot 配置限制单文件大小（max-file-size: 50MB）和请求大小（max-request-size: 100MB），避免超大文件攻击；通过预签名 URL + 流式上传优化内存占用，30MB 文件不会占用 100MB 堆内存。但这还不够。假设你的服务器磁盘只有 100GB 可用空间，某个时刻有 100 个用户同时上传 30MB 的文件，即使每个文件都合规，瞬间就要占用 3GB 临时磁盘空间。如果上传速度慢（比如用户网络差），这些临时文件会在磁盘上停留很久。更极端的情况，1000 个并发上传，磁盘 IO 直接被打满，其他正常请求也会受影响。这篇文章讲如何用分布式信号量限流来控制同时上传的并发数，以及为什么最终选择了 Redisson 的 RPermitExpirableSemaphore。

## 为什么还需要限流

### 1. 并发上传的风险

单文件大小限制和内存优化解决的是单个请求的问题，但没有解决并发场景下的资源竞争。

#### 1.1 磁盘空间膨胀

文件上传时，Spring Boot 会先把文件写到临时目录（java.io.tmpdir），处理完成后再移动到目标位置或上传到对象存储。如果同时有大量上传请求，临时目录会堆积大量文件。  
假设单文件限制 30MB，同时有 100 个上传请求，临时目录瞬间占用 3GB。如果用户网络慢，上传耗时 30 秒，这 3GB 空间要占用 30 秒才能释放。如果磁盘空间不足，新的上传请求会失败，甚至影响其他服务。  
更糟糕的是，如果某些上传请求因为网络问题或服务器异常卡住了，这些临时文件会一直占用磁盘空间，直到请求超时或被手动清理。

#### 1.2 磁盘 IO 飙升

大量并发写磁盘会导致 IO 飙升。机械硬盘的随机写性能本来就差，SSD 虽然好一些，但并发写入过多也会影响整体性能。磁盘 IO 打满后，不仅文件上传变慢，数据库查询、日志写入等其他操作也会受影响。你可能会看到：

- 数据库查询响应时间从几十毫秒飙升到几秒
- 应用日志写入延迟，监控告警延迟
- 其他需要读写磁盘的接口全部变慢

#### 1.3 对象存储带宽占用

即使文件不在本地磁盘停留太久，上传到对象存储（S3、OSS 等）也需要消耗网络带宽。如果同时有大量上传请求，服务器的出口带宽会被占满，影响其他正常业务。

### 2. 极端场景下的攻击风险

即使不是恶意攻击，用户行为也可能造成类似效果。比如某个企业客户写了个脚本批量上传文档，一次性发起 500 个并发请求。每个请求都合规（文件大小、格式都没问题），但服务器扛不住这么多并发。  
或者更隐蔽的场景：某个用户的网络很差，上传一个 30MB 文件要 5 分钟。如果他同时打开 10 个浏览器标签页上传文件，这 10 个请求会同时占用服务器资源 5 分钟。

## 为什么不用 QPS 限流

### 1. QPS 限流的语义

QPS（Queries Per Second）限流控制的是每秒允许通过的请求数。比如限制 10 QPS，意思是每秒最多放行 10 个请求。这种限流适合快速响应的接口，比如查询接口，一个请求几十毫秒就处理完了。限制 10 QPS，实际并发数也就是 10 左右。

#### 1.1 QPS 限流的工作原理

常见的 QPS 限流算法有：

- 固定窗口：每秒重置计数器，简单但有突刺问题
- 滑动窗口：更平滑，但实现复杂
- 令牌桶：允许短时突发，适合流量不均匀的场景
- 漏桶：严格控制速率，适合需要平滑输出的场景

但无论哪种算法，QPS 限流关注的都是请求的到达速率，而不是请求的持续时间。

## 2. 文件上传是长耗时操作

文件上传不一样。一个 30MB 的文件，用户网络如果是 1MB/s，上传就要 30 秒。如果限制 10 QPS：

- 第 1 秒：放行 10 个请求，这 10 个请求开始上传
- 第 2 秒：又放行 10 个请求，但第 1 秒的 10 个请求还在上传中
- 第 3 秒：再放行 10 个请求，前面 20 个请求还在上传中
- ...
- 第 30 秒：已经放行了 300 个请求，这 300 个请求都在同时上传

实际并发数远超预期，磁盘和 IO 照样被打满。

### 2.1 用数据说话

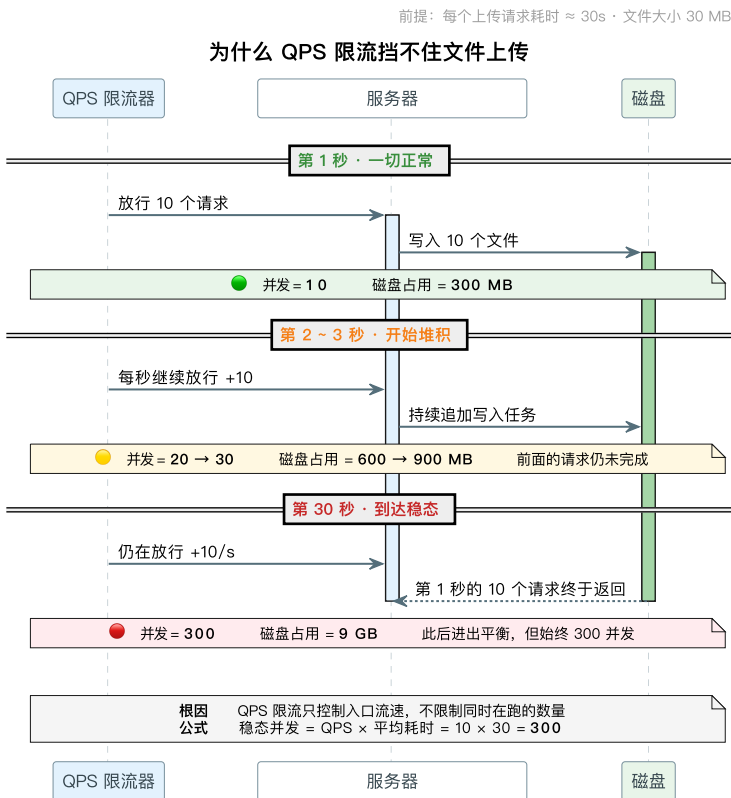
假设：

- 单文件限制 30MB
- 用户平均上传速度 1MB/s
- 平均上传耗时 30 秒
- QPS 限制 10

那么稳态下的并发数 = QPS × 平均耗时 = 10 × 30 = 300 个并发请求。  
如果每个请求占用 30MB 临时磁盘空间，300 个并发就是 9GB。这完全失控了。

### 2.2 QPS 限流失控示意图

下面用时序图展示 QPS 限流在文件上传场景下的问题：



## 3. 我们需要控制的是并发数

文件上传场景下，我们真正需要控制的是同时上传的请求数，而不是每秒的请求数。比如限制最多 10 个并发上传，那么：

- 前 10 个请求进来，开始上传
- 第 11 个请求进来，发现已经有 10 个在上传，等待
- 第 1 个请求上传完成，释放一个位置，第 11 个请求才能开始上传

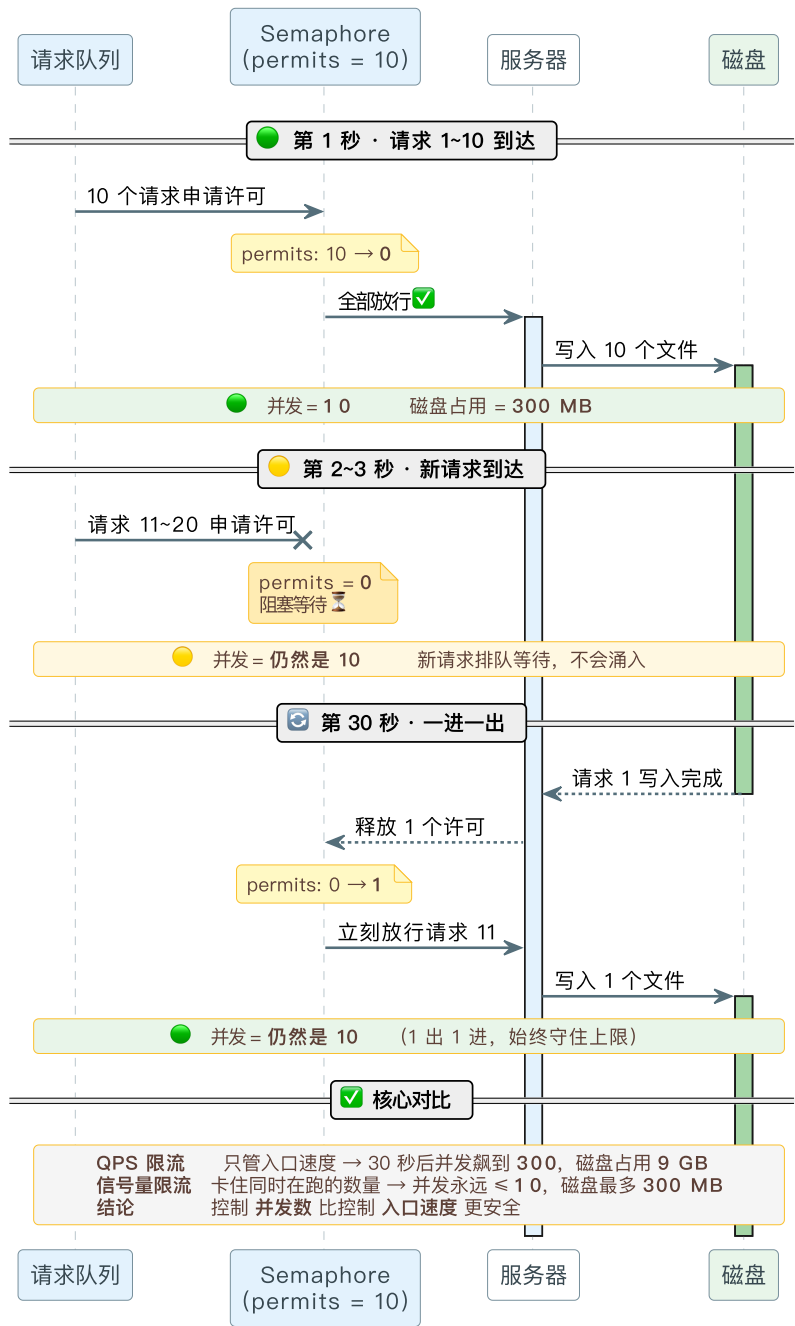
这样无论上传耗时多久，同时占用磁盘的请求数始终不超过 10 个。  
这正是信号量（Semaphore）的语义。

### 3.1 信号量限流示意图

下面用时序图展示信号量如何控制并发数：

前提：每个上传请求耗时  $\approx 30s$  · 文件大小 30 MB

信号量限流：精确控制并发数



信号量 Semaphore

1. 什么是信号量

信号量（Semaphore）是一种用于多线程编程的同步机制。它可以控制多个线程对共享资源的访问，确保在同一时间只有一定数量的线程能够访问该资源。信号量通常用于解决线程间的竞争问题，防止资源冲突，或者实现线程之间的通信。

1.1 信号量的工作原理

信号量内部维护了一个计数器，用于记录当前可被访问的资源数量。信号量的基本操作有两个：

- 1. `acquire()`：尝试从信号量中获取一个许可。如果信号量的计数器大于零，则计数器减一，并允许线程继续执行。如果计数器为零，则线程进入等待状态，直到有其他线程释放一个许可。
- 2. `release()`：释放一个许可，将信号量的计数器加一。如果有线程在等待信号量，则唤醒其中的一个线程。

比如创建一个许可数为 3 的信号量，最多允许 3 个线程同时访问资源。第 4 个线程调用 `acquire()` 时会阻塞，直到前面某个线程调用 `release()` 释放许可。

1.2 信号量的应用场景

信号量特别适合控制对有限资源的并发访问，比如：

- 数据库连接池：限制同时使用的数据库连接数
- 线程池：限制同时执行的任务数
- 文件上传：限制同时上传的文件数（我们的场景）
- API 限流：限制同时调用某个 API 的请求数

2. Java 原生信号量示例

在 Java 中，信号量由 `java.util.concurrent.Semaphore` 类实现。下面是一个简单的示例：

```
import java.util.concurrent.Semaphore;

public class SemaphoreExample {

    // 创建一个具有 3 个许可的信号量
    private static final Semaphore semaphore = new Semaphore(3);

    public static void main(String[] args) {
        // 创建 10 个线程模拟并发访问
        for (int i = 0; i < 10; i++) {
            new Thread(new Task(), "Thread-" + i).start();
        }
    }

    static class Task implements Runnable {
        @Override
        public void run() {
            try {
                // 尝试获取一个许可
                semaphore.acquire();
                System.out.println(Thread.currentThread().getName() + " 获取许可，开始上传");

                // 模拟文件上传耗时
                Thread.sleep(2000);

                System.out.println(Thread.currentThread().getName() + " 上传完成");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                // 释放许可
                semaphore.release();
                System.out.println(Thread.currentThread().getName() + " 释放许可");
            }
        }
    }
}
```

运行这段代码，你会看到最多只有 3 个线程同时在上传，其他线程在等待。输出类似：

```
Thread-0 获取许可，开始上传
Thread-1 获取许可，开始上传
Thread-2 获取许可，开始上传
Thread-0 上传完成
Thread-0 释放许可
Thread-3 获取许可，开始上传
...
```

关键点说明：

- 必须在 finally 块中释放许可：**即使上传过程中抛出异常，也要确保许可可被释放，否则会导致许可永久丢失。
- 公平性选项：**`Semaphore` 构造函数有一个 `fair` 参数。如果设置为 `true`，信号量会按照线程请求的顺序分配许可（FIFO），避免线程饥饿。但公平模式会有性能开销。

可中断的等待：acquire() 方法会响应中断。如果线程在等待许可时被中断，会抛出 InterruptedException。

### 3. 局限性

Java 原生的 Semaphore 是 JVM 级别的，只能在单个应用实例内生效。如果你的服务部署了多个实例（比如 3 台服务器做负载均衡），每个实例都有自己的信号量，互不影响。

假设每个实例的信号量许可数是 10，3 个实例总共可以同时处理 30 个上传请求。如果你想在整个集群层面限制并发数为 10，本地信号量做不到。

**为什么需要分布式信号量？**

在微服务架构或多实例部署的场景下，我们需要一个跨实例共享的信号量。这就需要把信号量的状态存储在一个中心化的地方，比如 Redis。

## Redisson 分布式信号量 RSemaphore

### 1. 基本原理

RSemaphore 是 Redisson 提供的分布式信号量实现，它基于 Redis 实现了信号量的功能，允许在分布式环境下控制对共享资源的访问。RSemaphore 是 Redisson 的一部分，用于在多个分布式应用实例之间协调对资源的访问。

#### 1.1 核心特性

**分布式信号量：**RSemaphore 是分布式的，这意味着它可以跨多个 Redis 节点和多个应用实例工作。在分布式环境中，它帮助协调对共享资源的访问。

**功能与 Java 原生信号量类似：**RSemaphore 提供了与 Java 原生 Semaphore 类似的功能，包括许可的获取和释放，但它在分布式环境下工作。

#### 1.2 工作原理

RSemaphore 的计数器存储在 Redis 中，多个应用实例共享同一个计数器，从而实现集群级别的并发控制。

工作原理和本地信号量一样，只是把计数器从 JVM 内存搬到了 Redis。多个实例调用 acquire() 时，都是在操作同一个 Redis key 的值。Redisson 使用 Lua 脚本保证操作的原子性。

### 2. 主要操作方法

acquire()：尝试获取一个许可。如果没有可用的许可，调用线程会被阻塞直到许可变得可用。

tryAcquire()：尝试获取一个许可，如果立即可以获取则返回 true，否则返回 false。

release()：释放一个许可，将其返回到信号量中。

availablePermits()：返回当前可用的许可数量。

### 3. 示例代码

```
import org.redisson.api.RSemaphore;
import org.redisson.api.RedissonClient;
import org.redisson.Redisson;
import org.redisson.config.Config;

public class RedissonSemaphoreExample {

    public static void main(String[] args) throws InterruptedException {
        // 配置 Redisson 客户端
        Config config = new Config();
        config.useSingleServer().setAddress("redis://127.0.0.1:6379");
        RedissonClient redisson = Redisson.create(config);

        // 获取分布式信号量
        RSemaphore semaphore = redisson.getSemaphore("mySemaphore");

        // 初始化信号量许可数量（只需要执行一次）
        semaphore.trySetPermits(10);

        // 获取许可
        semaphore.acquire();
        System.out.println("获取许可，开始上传");
    }
}
```

```
// 模拟上传
Thread.sleep(5000);

// 释放许可
semaphore.release();
System.out.println("上传完成，释放许可");

// 关闭 Redisson 客户端
redisson.shutdown();
}
}
```

#### 代码说明：

1. **初始化许可数量**：trySetPermits(10) 只在第一次调用时生效，后续调用不会改变许可数量。这个方法是幂等的。
2. **阻塞获取**：acquire() 会阻塞当前线程，直到获取到许可。如果不想阻塞，可以使用 tryAcquire() 或带超时的 tryAcquire(timeout, unit)。
3. **释放许可**：release() 释放一个许可。注意，Redisson 的 RSemaphore 不会检查释放许可的线程是否是获取许可的线程，这和 Java 原生的 Semaphore 一样。

## 4. 宕机问题

RSemaphore 看起来完美解决了分布式场景的问题，但有一个致命缺陷：如果某个实例获取了许可，但在释放之前宕机了，这个许可就永久丢失了。

### 4.1 问题场景

举个例子：

1. 信号量初始许可数是 10
2. 实例 A 调用 acquire() 获取了一个许可，Redis 中的计数器从 10 变成 9
3. 实例 A 开始处理上传，但处理到一半时服务器宕机
4. 实例 A 没有机会调用 release()，Redis 中的计数器永远停留在 9
5. 如果这种情况发生多次，许可数会不断减少：9 → 8 → 7 → ...
6. 最终许可数变成 0，所有上传请求都会被阻塞

### 4.2 为什么不可接受

这在生产环境是不可接受的。你不能因为偶尔的宕机就让系统的并发能力永久下降。

想象一下，你的系统配置了 10 个并发上传。运行一段时间后，因为各种原因（服务器重启、进程崩溃、网络断开）丢失了 5 个许可，实际并发能力变成了 5。再过一段时间，又丢失了 3 个许可，实际并发能力变成了 2。

最终，你的系统几乎无法处理上传请求，但你可能根本不知道是信号量许可丢失导致的。

### 4.3 可能的解决方案

有人可能会想到几种解决方案：

**定期重置许可数**：定时任务每隔一段时间重置许可数为 10。但这会导致短时间内并发数超过限制。

**心跳机制**：持有许可的实例定期发送心跳，超时未心跳则回收许可。但这增加了复杂度，而且心跳本身也可能失败。

**使用 Redis 过期时间**：给许可设置过期时间。这就是 RPermitExpirableSemaphore 的思路。

## Redisson 可过期信号量 RPermitExpirableSemaphore

### 1. 核心特性

RPermitExpirableSemaphore 是 Redisson 提供的一个分布式信号量实现，它支持许可的过期功能。与标准的 RSemaphore 不同，RPermitExpirableSemaphore 允许许可在一定时间后自动过期，从而增强了对资源访问的控制能力，适用于需要许可自动失效的场景。

#### 1.1 过期许可的优势

RPermitExpirableSemaphore 允许许可在一定时间后自动过期，避免了因进程异常终止或其他问题导致许可无法释放的情况。这有助于确保系统资源不会因许可长时间未释放而耗尽。

这个特性完美解决了宕机问题：

**正常情况下**：处理完成后主动调用 release() 释放许可

异常情况下（宕机、网络断开等）：许可到期后自动释放，不会永久占用

## 1.2 分布式与自动释放

**分布式信号量**：RPermitExpirableSemaphore 是分布式的，基于 Redis 实现，使其能够在多个分布式实例之间协调对资源的访问。

**支持自动释放**：许可会在设定的时间后自动释放，无需手动干预，适用于对资源访问有时间限制的场景。

因为它具备自动释放特性，非常符合我们的需求。也就意味着，即使你宕机但是依然能释放。

## 2. 主要操作方法

`acquire(long leaseTime, TimeUnit unit)`：同步阻塞获取许可，并设置许可的有效期。如果许可数量不足，线程将阻塞，直到有足够的许可可用。返回一个 `permitId`。

`tryAcquire(long waitTime, long leaseTime, TimeUnit unit)`：尝试在指定时间内获取指定数量的许可，并设置许可的有效期。如果无法在超时时间内获取到足够的许可，则返回 `null`。

`release(String permitId)`：释放指定的许可。需要传入获取许可时返回的 `permitId`。

`availablePermits()`：返回当前可用的许可数量。

## 3. 示例代码

```
import org.redisson.api.RPermitExpirableSemaphore;
import org.redisson.api.RedissonClient;
import org.redisson.Redisson;
import org.redisson.config.Config;

import java.util.concurrent.TimeUnit;

public class RedissonPermitExpirableSemaphoreExample {

    public static void main(String[] args) throws InterruptedException {
        // 配置 Redisson 客户端
        Config config = new Config();
        config.useSingleServer().setAddress("redis://127.0.0.1:6379");
        RedissonClient redisson = Redisson.create(config);

        // 获取分布式信号量
        RPermitExpirableSemaphore semaphore =
            redisson.getPermitExpirableSemaphore("myExpirableSemaphore");

        // 初始化信号量许可数量
        semaphore.trySetPermits(10);

        // 尝试获取许可，设置有效期为 10 秒
        String permitId = semaphore.tryAcquire(5, 10, TimeUnit.SECONDS);

        if (permitId != null) {
            System.out.println("获取许可成功, permitId=" + permitId);

            try {
                // 模拟文件上传
                Thread.sleep(3000);
                System.out.println("上传完成");
            } finally {
                // 主动释放许可
                semaphore.release(permitId);
                System.out.println("许可已释放");
            }
        } else {
            System.out.println("获取许可失败, 当前上传人数过多");
        }

        // 关闭 Redisson 客户端
        redisson.shutdown();
    }
}
```

```
    }  
}
```

3.1 关键点说明

- 1. **tryAcquire 的三个参数：**
  - `waitTime`：最大等待时间。如果当前没有可用许可，最多等待这么久
  - `leaseTime`：许可的有效期。获取到许可后，如果在这个时间内没有主动释放，许可会自动过期
  - `unit`：时间单位
- 2. **返回值是 `permitId`**： `tryAcquire()` 返回一个字符串类型的 `permitId`，释放时需要传入这个 ID。如果获取失败（等待超时），返回 `null`。
- 3. **必须用 `permitId` 释放**： `release(permitId)` 需要传入获取许可时返回的 ID。这样可以确保只有持有许可的实例才能释放它。
- 4. **自动过期机制**：如果在 `leaseTime` 时间内没有调用 `release()`，许可会自动过期释放。这是和 `RSemaphore` 最大的区别。

4. 为什么适合文件上传场景

文件上传有明确的时间上限。比如我们限制单文件 30MB，用户网络再慢，5 分钟也足够上传完了。如果 5 分钟还没完成，要么是网络异常，要么是服务器处理异常，这个请求应该被超时中断。

4.1 时间上限的合理性

基于这个特点，我们可以把许可的有效期设置为 5 分钟（按照具体的企业上传大小和网速等设置）：

- 正常情况**：上传在 1-2 分钟内完成，主动释放许可
- 异常情况**：服务器宕机或网络断开，5 分钟后许可自动过期释放
- 恶意占用**：即使有人故意获取许可后不释放，5 分钟后也会自动释放

这样既保证了并发控制，又避免了许可永久丢失的问题。

4.2 过期时间的设置策略

过期时间的设置需要权衡：

- 太短**：正常上传可能还没完成就过期了，导致许可被回收但请求还在处理
- 太长**：异常情况下许可占用时间过长，影响系统吞吐量

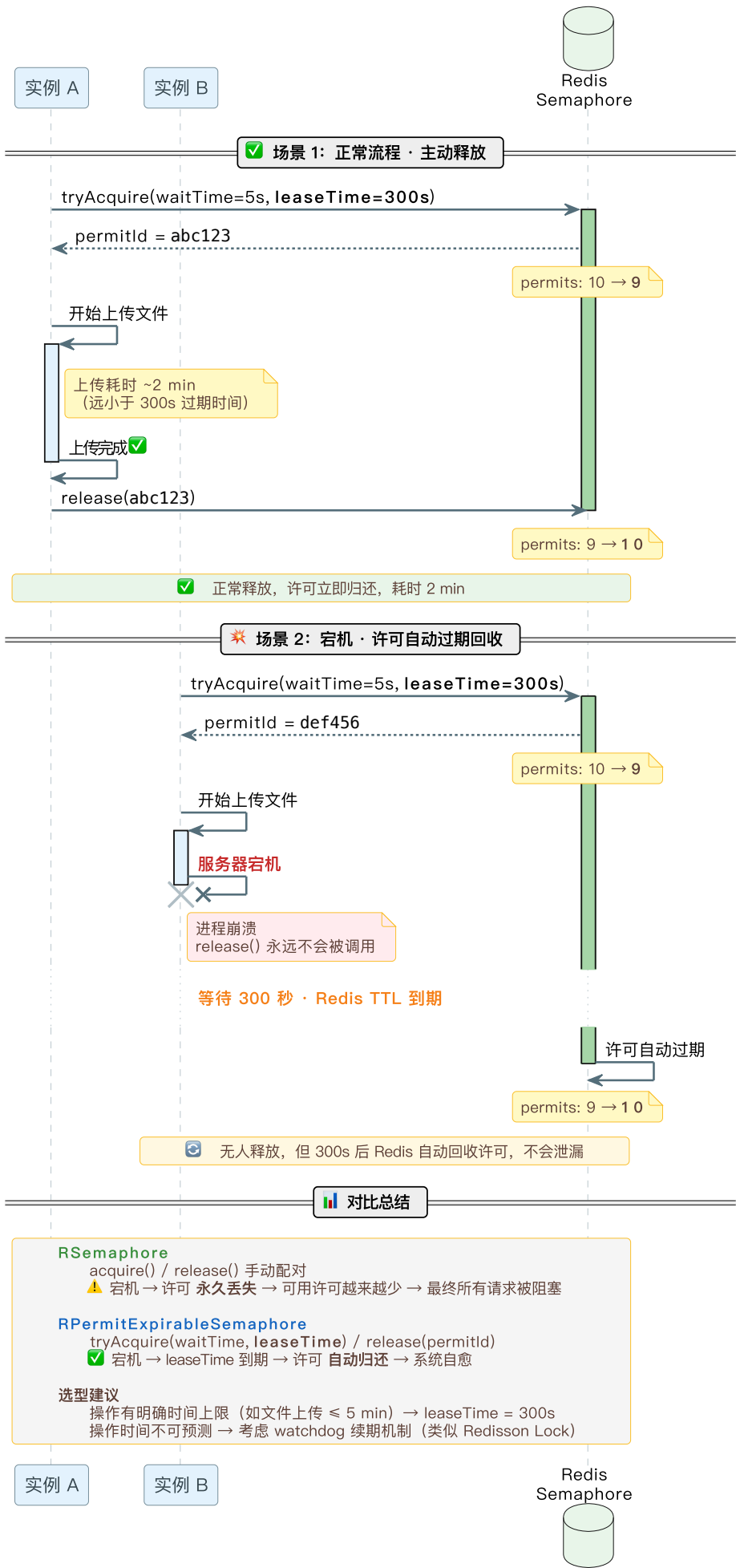
建议设置为正常上传时间的 2-3 倍。比如正常上传耗时 1-2 分钟，过期时间设置为 5 分钟。

4.3 RPermitExpirableSemaphore 工作流程图

下面用时序图展示可过期信号量的两种场景：正常释放和自动过期。



可过期信号量：正常释放 vs 宕机自动回收



## Ragent 项目中的实现

### 1. 配置设计

#### 1.1 配置类

RagSemaphoreProperties 定义了信号量的配置参数：

```
@Data
@Validated
@Configuration
@ConfigurationProperties(prefix = "rag.semaphore")
public class RagSemaphoreProperties {

    @Valid
    private PermitExpirableConfig documentUpload = new PermitExpirableConfig();

    @Data
    public static class PermitExpirableConfig {

        /**
         * Redisson 信号量名称
         */
        @NotBlank
        private String name = "rag:document:upload";

        /**
         * 最大并发数
         */
        @Min(1)
        private Integer maxConcurrent = 10;

        /**
         * 获取许可最大等待时间（秒）
         */
        @Min(0)
        private Integer maxWaitSeconds = 30;

        /**
         * permit 自动释放时间（秒）
         */
        @Min(1)
        private Integer leaseSeconds = 30;
    }
}
```

#### 1.2 配置项说明

在 application.yaml 中配置：

```
rag:
  semaphore:
    document-upload:
      name: rag:document:upload      # Redis key 名称
      max-concurrent: 10             # 最多 10 个并发上传
      max-wait-seconds: 5            # 等待许可最多 5 秒
      lease-seconds: 300             # 许可有效期 5 分钟
```

配置参数的含义：

- name：Redis 中存储信号量的 key 名称。不同的业务场景可以使用不同的 key，实现隔离。
- max-concurrent：整个集群同时允许的最大上传并发数。这是核心参数，需要根据服务器资源（磁盘空间、IO 能力、网络带宽）来设置。

`max-wait-seconds`：用户请求进来后，如果当前并发已满，最多等待多久。超时后返回“当前上传人数过多，请稍后再试”。这个值不宜太长，否则用户体验差。

`lease-seconds`：许可的有效期。正常情况下上传完成会主动释放，异常情况下到期自动释放。这个值应该设置为正常上传时间的 2–3 倍。

### 1.3 配置参数的权衡

`max-concurrent` 的设置：

太小：系统吞吐量低，用户经常等待

太大：磁盘和 IO 压力大，可能影响其他服务

建议根据压测结果设置。比如单个上传请求平均占用 30MB 临时磁盘空间，服务器有 10GB 可用空间，理论上可以支持 300+ 并发。但考虑到 IO 瓶颈，实际可能只能支持 50–100 并发。

`lease-seconds` 的设置：

太短：正常上传可能还没完成就过期，导致许可被回收但请求还在处理

太长：异常情况下许可占用时间过长

建议设置为 P99 上传时间的 2 倍。比如 99% 的上传请求在 2 分钟内完成，设置为 5 分钟（300 秒）。

## 2. 信号量初始化

`SemaphoreInitializer` 在应用启动时初始化信号量：

```
@Slf4j
@Component
@RequiredArgsConstructor
public class SemaphoreInitializer {

    private final RedissonClient redissonClient;
    private final RagSemaphoreProperties semaphoreProperties;

    @PostConstruct
    public void documentUploadSemaphoreInitialize() {
        RagSemaphoreProperties.PermitExpirableConfig config = semaphoreProperties.getDocumentUploadPermitExpirableSemaphore
        RPermitExpirableSemaphore semaphore = redissonClient.getPermitExpirableSemaphore(config.getName());

        semaphore.setPermits(config.getMaxConcurrent());
        log.info("Initialized document upload semaphore: name={}, maxConcurrent={}",
            config.getName(),
            config.getMaxConcurrent());
    }
}
```

### 2.1 初始化逻辑说明

这里有两个关键操作：

1. **根据配置获取信号量**：咱们当前是只有文档上传的信号量，但是接下来会更多。
2. **对齐许可总数**：`setPermits()` 设置信号量的总许可数。如果配置改了（比如从 10 改成 20），重启后会自动对齐。

### 2.2 为什么需要初始化

`RPermitExpirableSemaphore` 的许可数不会自动初始化，需要手动设置。如果不初始化，`availablePermits()` 会返回 0，所有请求都会被阻塞。

另外，如果修改了配置文件中的 `max-concurrent`，重启后需要重新设置许可数。`setPermits()` 会覆盖 Redis 中的旧值。

## 3. 上传方法中的限流逻辑

### 3.1 获取和释放许可的封装

在 `KnowledgeDocumentServiceImpl` 中封装了获取和释放许可的方法：

```

private String acquireUploadPermit(RPermitExpirableSemaphore semaphore,
                                   RagSemaphoreProperties.PermittableConfig config) {
    try {
        String permitId = semaphore.tryAcquire(
            config.getMaxWaitSeconds(),
            config.getLeaseSeconds(),
            TimeUnit.SECONDS
        );
        if (StringUtil.isBlank(permitId)) {
            throw new ClientException("当前上传文件人数过多，请稍后再试");
        }
        return permitId;
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt();
        throw new ServiceException("获取上传并发许可失败");
    }
}

private void releaseUploadPermit(RPermitExpirableSemaphore semaphore, String permitId) {
    if (StringUtil.isNotBlank(permitId)) {
        boolean released = semaphore.tryRelease(permitId);
        if (!released) {
            log.warn("upload permit already expired or released, permitId={}", permitId);
        }
    }
}
}

```

### 3.2 封装的好处

**统一异常处理：**获取许可失败时，抛出友好的业务异常，前端可以展示“当前上传人数过多，请稍后再试”。

**处理中断异常：**tryAcquire() 可能抛出 InterruptedException，需要正确处理。这里恢复中断状态并抛出业务异常。

**释放失败的容错：**tryRelease() 可能返回 false（许可已过期或已释放）。这种情况下只记录警告日志，不影响主流程。

### 3.3 在上传方法中使用

在文档上传方法中，使用 try-finally 结构确保许可一定会被释放：

```

@Override
public KnowledgeDocumentVO upload(Long kbId,
                                   MultipartFile file,
                                   KnowledgeDocumentUploadRequest request) {

    // ... 参数校验和知识库查询 ...

    // 获取信号量配置
    RagSemaphoreProperties.PermittableConfig semaphoreConfig =
        semaphoreProperties.getDocumentUpload();
    RPermitExpirableSemaphore semaphore =
        redissonClient.getPermittableSemaphore(semaphoreConfig.getName());

    // 获取许可
    String permitId = acquireUploadPermit(semaphore, semaphoreConfig);

    try {
        // 上传文件到对象存储
        StoredFileDTO stored = resolveStoredFile(
            kbDO.getCollectionName(),
            sourceType,
            sourceLocation,
            file
        );

        // ... 后续处理：保存文档记录、发送分块消息等 ...
    }
}

```

```
        return BeanUtil.toBean(documentDO, KnowledgeDocumentV0.class);
    } finally {
        // 无论成功还是失败，都释放许可
        releaseUploadPermit(semaphore, permitId);
    }
}
```

3.4 关键点说明

- 1. 在文件上传之前获取许可：如果获取失败，直接返回错误，不会执行后续的文件上传逻辑。这样可以避免无效的资源占用。
- 2. 使用 try-finally 确保许可释放：即使上传过程中抛出异常，finally 块也会执行，释放许可。这是防止许可泄漏的关键。
- 3. 许可的生命周期：

- 获取许可：在文件上传开始之前
- 持有许可：整个文件上传和处理过程
- 释放许可：文件处理完成或异常退出时

4. 异常情况的处理：

- 如果获取许可超时，抛出 ClientException，前端展示友好提示
- 如果上传过程中出现异常，finally 块确保许可可被释放
- 如果服务器宕机，许可会在 leaseSeconds 后自动过期

小结

文件上传的并发控制需要考虑三个层面的问题：

为什么需要限流：

单文件大小限制和内存优化解决了单个请求的问题，但大量并发上传仍会导致：

- 磁盘空间膨胀：临时文件堆积
- 磁盘 IO 飙升：影响其他服务
- 对象存储带宽占用：网络资源耗尽

为什么不用 QPS 限流：

QPS 限流控制每秒请求数，但文件上传是长耗时操作。限制 10 QPS，稳态下的实际并发数 = 10 × 平均耗时（秒）。如果平均耗时 30 秒，实际并发数是 300，完全失控。  
我们需要的是信号量语义：控制同时上传的请求数，而不是每秒的请求数。

选型推演过程：

Java 本地 Semaphore：

- 优点：简单易用，性能好
- 缺点：只能在单实例内生效，无法跨集群

Redisson RSemaphore：

- 优点：支持分布式，跨实例共享计数器
- 缺点：宕机会导致许可永久丢失，系统并发能力不可恢复地下降

Redisson RPermitExpirableSemaphore：

- 优点：许可带过期时间，宕机后自动释放，适合文件上传场景
- 缺点：需要合理设置过期时间，过短会导致正常请求被中断，过长会影响异常恢复速度

项目实现要点：

- 配置类：定义并发数、等待时间、过期时间
- 初始化器：启动时初始化信号量许可数，触发过期许可清理
- 上传方法：使用 try-finally 确保许可释放，即使异常也不会泄漏
- 异常处理：获取许可失败时抛出友好的业务异常，释放失败时记录警告日志

通过分布式可过期信号量，我们在整个集群层面控制了文件上传的并发数，既保护了磁盘资源，又避免了许可永久丢失的问题。这是一个在可靠性和性能之间取得平衡的方案。

